A CASE-STUDY BASED REPORT

ON

Fraudulent Transaction Clustering Using DBSCAN-Based Anomaly Detection

submitted in partial fulfilment of the requirements for the Course

MACHINE LEARNING

BACHELOR OF TECHNOLOGY

in

A.Y.: 2026-27

by

Arikela Sumanjali **2520090190**

Gatik Sharma **2520090223**

Under the Esteemed guidance of

Dr.Y.Lakshmi Prasanna

Asst.Professor, Dept. of CS&IT



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

Abstract

Financial fraud is a critical challenge in the digital characterized by corustantly evolving tactics tynt often traditional rule-based and supervised detection Systems. This project explores the application Of unsucxrvisai nuchine kaming. fically clustering algorithms, to identify fraudulent transactions without the need for extensively By grouping transactions based on behavioral and transactional similarities, the model establishes a baseline Of activity. Outliers and transactions that fall outside these dense clusters are subsequently flagged as Bitential fraud. This approach offers a dynamic and scalable solution capable Of adapting to novel, previously unseen fraud patterns, ultimately enhancing the security and anomaly detection capabilities of mcdern financial systems.

Introduction

As digital payment systems online banking expand. financial institutions face an surge in tramacticm accompanied by a increase in fraudulent activities. Fraudsters continuously adapt their traditional, static rule-based systems ineffective arui prone to generating overwhelming numbers of false p«itives. While sulkrvised machine learning have improved detection rates, they heavily rely on historically labeled datil This makes them highly vulnerable to "zero-day" or antirely new fraud techniques that have not yet categorind. To address this critical limitation, this project an unsuFrvised machine learning approach leveraging techniques. Unlike suEMVised models, clustering algorithms—such as K-Means, DBSCAN, or Hierarchical not require pre-labcled data to function, lnstead. they analyze the inherent structure Of the dataset, grouping transxtions based on slured features such as transaction amount. geographical "ration. time, and frequency. By mathematically defining what constitutes "nontur customer behavior through these clusters. any anomaW transmions that deviate significantly from established patterns can isolatd This study aims to implement. visualize. and evaluate Clustering models to demonstrate their efficacy in detecting complex, evolving financial fraud in a highly irinlanced data environment.

Problem Statement

Case Study Question

A financial institution processes a large number of digital transactions every day. Due to the increasing number of fraudulent activities, the institution wants to identify unusual transaction patterns without relying on predefined fraud rules or using the fraud labels during detection. Using the given PaySim transaction dataset, perform Exploratory Data Analysis (EDA) to understand transaction behavior and then apply the DBSCAN clustering algorithm to identify anomalous transactions. Determine whether the transactions identified as DBSCAN outliers correspond to actual fraudulent transactions by comparing the detected anomalies with the isFraud field after clustering.

Dataset and Exploratory Data Analysis

1.DATASET DESCRIPTION

The **PaySim dataset** is a synthetic financial transaction dataset used for studying fraud detection. It contains **6,362,620 transactions and 11 attributes** describing transaction details, account balances, and fraud indicators.

Important fields include:

- **step** – Transaction time step.
- **type** – Type of transaction such as PAYMENT, TRANSFER, CASH_OUT, etc.

- **amount** – Transaction amount.
- **oldbalanceOrg / newbalanceOrig** – Sender's balance before and after the transaction.
- **oldbalanceDest / newbalanceDest** – Receiver's balance before and after the transaction.
- **isFraud** – Actual fraud indicator (0 = normal, 1 = fraud).

For DBSCAN, relevant transaction features are selected and scaled, while isFraud is kept only for evaluating the detected anomalies after clustering.

2.EXPLORATORY DATA ANALYSIS
Dataset loaded successfully!

First 5 Records:

	step	type	amount	nameOrig	oldbalanceOrg	newbalanceOrig	nameDest	oldbalanceDest	newbalanceDest	isFraud	isFlaggedFraud
0	1	PAYMENT	9839.64	C1231006815	170136.0	160296.36	M1979787155	0.0	0.0	0	0
1	1	PAYMENT	1864.28	C1666544295	21249.0	19384.72	M2044282225	0.0	0.0	0	0
2	1	TRANSFER	181.00	C1305486145	181.0	0.00	C553264065	0.0	0.0	1	0
3	1	CASH_OUT	181.00	C840083671	181.0	0.00	C38997010	21182.0	0.0	1	0
4	1	PAYMENT	11668.14	C2048537720	41554.0	29885.86	M1230701703	0.0	0.0	0	0

Last 5 Records:

	step	type	amount	nameOrig	oldbalanceOrg	newbalanceOrig	nameDest	oldbalanceDest	newbalanceDest	isFraud	isFlaggedFraud
6362615	743	CASH_OUT	339682.13	C786484425	339682.13	0.0	C776919290	0.00	339682.13	1	0
6362616	743	TRANSFER	6311409.28	C1529008245	6311409.28	0.0	C1881841831	0.00	0.00	1	0
6362617	743	CASH_OUT	6311409.28	C1162922333	6311409.28	0.0	C1365125890	68488.84	6379898.11	1	0
6362618	743	TRANSFER	850002.52	C1685995037	850002.52	0.0	C2080388513	0.00	0.00	1	0
6362619	743	CASH_OUT	850002.52	C1280323807	850002.52	0.0	C873221189	6510099.11	7360101.63	1	0

Dataset Shape:

Rows : 6362620

Columns : 11

Column Names:

['step', 'type', 'amount', 'nameOrig', 'oldbalanceOrg', 'newbalanceOrig', 'nameDest', 'oldbalanceDest', 'newbalanceDest', 'isFraud', 'isFlaggedFraud']

Data Types:

step int64

type object

amount float64

nameOrig object

oldbalanceOrg float64

```
newbalanceOrig    float64
nameDest          object
oldbalanceDest    float64
newbalanceDest    float64
isFraud           int64
isFlaggedFraud    int64
dtype: object

Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6362620 entries, 0 to 6362619
Data columns (total 11 columns):
 #   Column          Dtype
---  ------          -----
 0   step            int64
 1   type            object
 2   amount          float64
 3   nameOrig        object
 4   oldbalanceOrg   float64
 5   newbalanceOrig  float64
 6   nameDest        object
 7   oldbalanceDest  float64
 8   newbalanceDest  float64
 9   isFraud         int64
 10  isFlaggedFraud  int64
dtypes: float64(5), int64(3), object(3)
memory usage: 534.0+ MB

Missing Values:
```

step 0

type 0

amount 0

nameOrig 0

oldbalanceOrg 0

newbalanceOrig 0

nameDest 0

oldbalanceDest 0

newbalanceDest 0

isFraud 0

isFlaggedFraud 0

dtype: int64

Number of Duplicate Records:

0

Statistical Summary:

	step	amount	oldbalanceOrg	newbalanceOrig	oldbalanceDest	newbalanceDest	isFraud	isFlaggedFraud
count	6.362620e+06	6.362620e+06	6.362620e+06	6.362620e+06	6.362620e+06	6.362620e+06	6.362620e+06	6.362620e+06
mean	2.433972e+02	1.798619e+05	8.338831e+05	8.551137e+05	1.100702e+06	1.224996e+06	1.290820e-03	2.514687e-06
std	1.423320e+02	6.038582e+05	2.888243e+06	2.924049e+06	3.399180e+06	3.674129e+06	3.590480e-02	1.585775e-03
min	1.000000e+00	0.000000e+00	0.000000e+00	0.000000e+00	0.000000e+00	0.000000e+00	0.000000e+00	0.000000e+00
25%	1.560000e+02	1.338957e+04	0.000000e+00	0.000000e+00	0.000000e+00	0.000000e+00	0.000000e+00	0.000000e+00

	step	amount	oldbalanceOrg	newbalanceOrig	oldbalanceDest	newbalanceDest	isFraud	isFlaggedFraud
50%	2.390000e+02	7.487194e+04	1.420800e+04	0.000000e+00	1.327057e+05	2.146614e+05	0.000000e+00	0.000000e+00
75%	3.350000e+02	2.087215e+05	1.073152e+05	1.442584e+05	9.430367e+05	1.111909e+06	0.000000e+00	0.000000e+00
max	7.430000e+02	9.244552e+07	5.958504e+07	4.958504e+07	3.560159e+08	3.561793e+08	1.000000e+00	1.000000e+00

Number of Unique Values:

step 743

type 5

amount 5316900

nameOrig 6353307

oldbalanceOrg 1845844

newbalanceOrig 2682586

nameDest 2722362

oldbalanceDest 3614697

newbalanceDest 3555499

isFraud 2

isFlaggedFraud 2

dtype: int64

Transaction Types:

type

CASH_OUT 2237500

PAYMENT 2151495

CASH_IN 1399284

TRANSFER 532909

DEBIT 41432

Name: count, dtype: int64

Fraud Distribution:

isFraud

0 6354407

1 8213

Name: count, dtype: int64

Fraud Percentage:

0.1291%

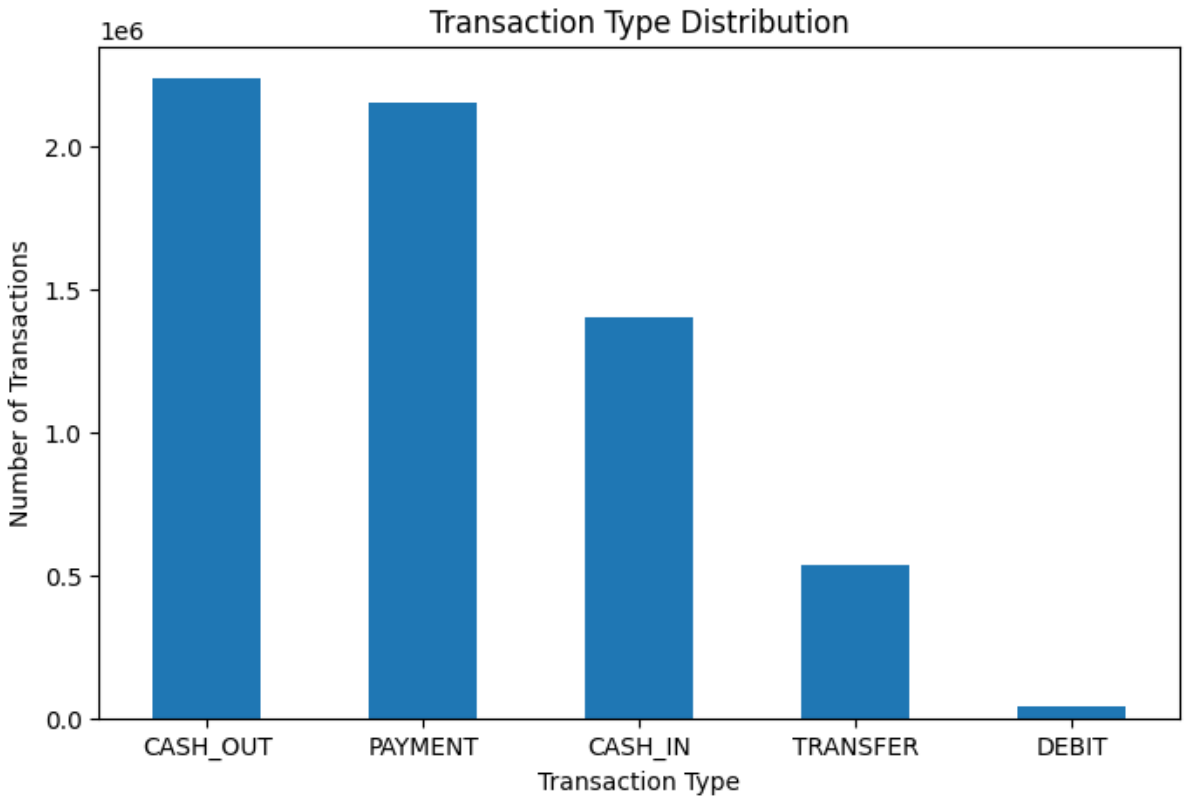
Flagged Fraud Distribution:

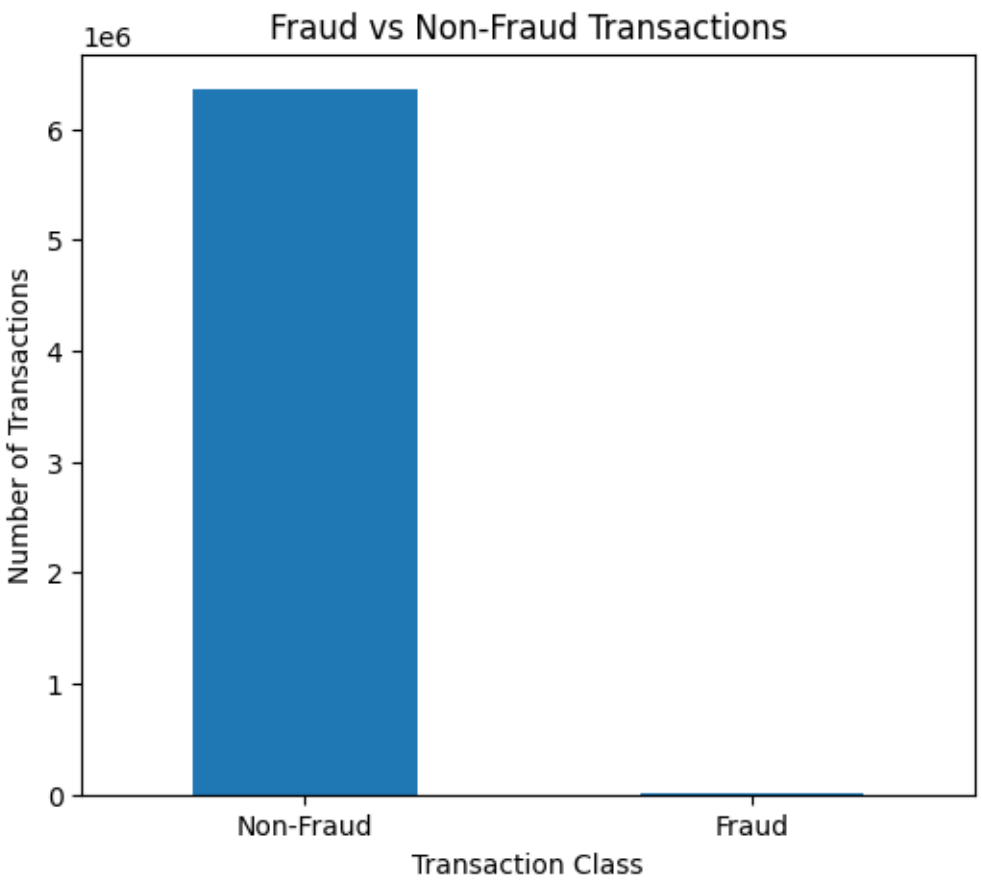
isFlaggedFraud

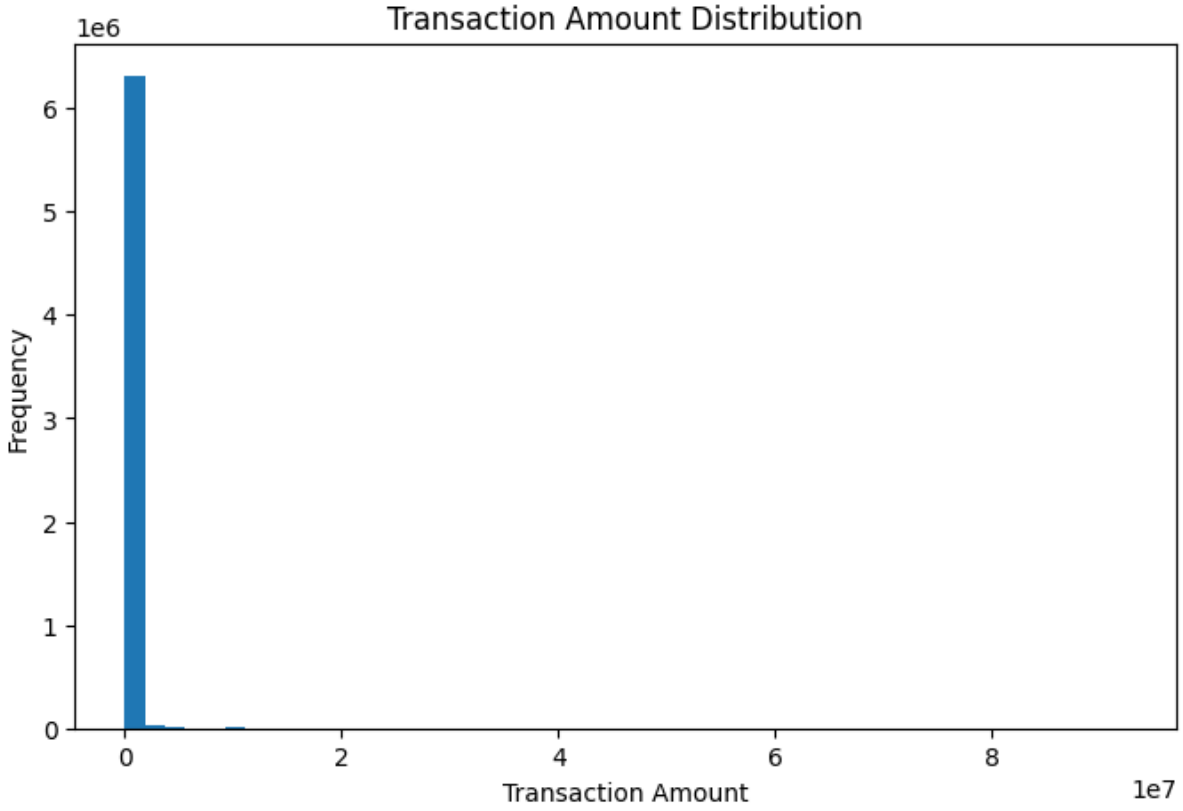
0 6362604

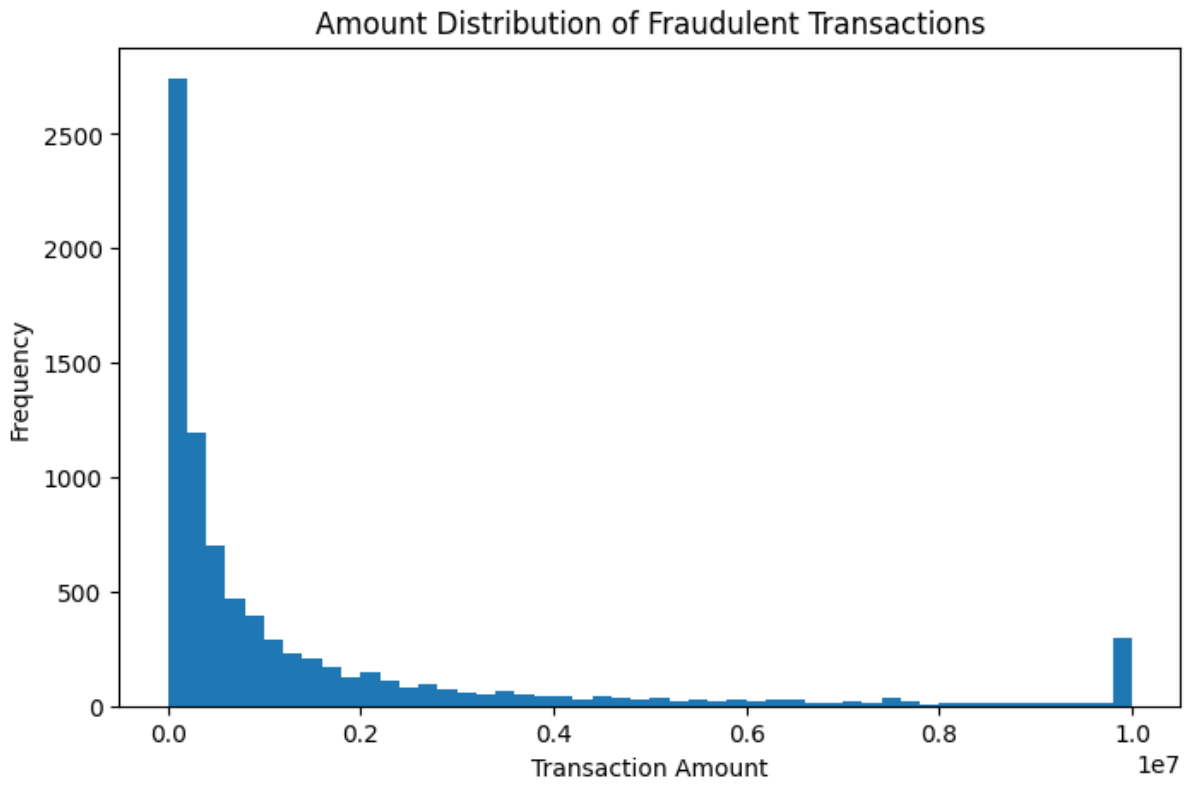
1 16

Name: count, dtype: int64




Fraud vs Non-Fraud Transactions
1e6
6
5
4
3
2
1
0
Number of Transactions
Non-Fraud
Fraud
Transaction Class


Transaction Amount Distribution
1e6
6
5
4
3
2
1
0
Frequency
0
2
4
6
8
Transaction Amount
1e7



Fraud by Transaction Type:

isFraud	0	1
type		
CASH_IN	1399284	0
CASH_OUT	2233384	4116
DEBIT	41432	0
PAYMENT	2151495	0
TRANSFER	528812	4097

Fraud Percentage by Transaction Type:

type

CASH_IN 0.000000

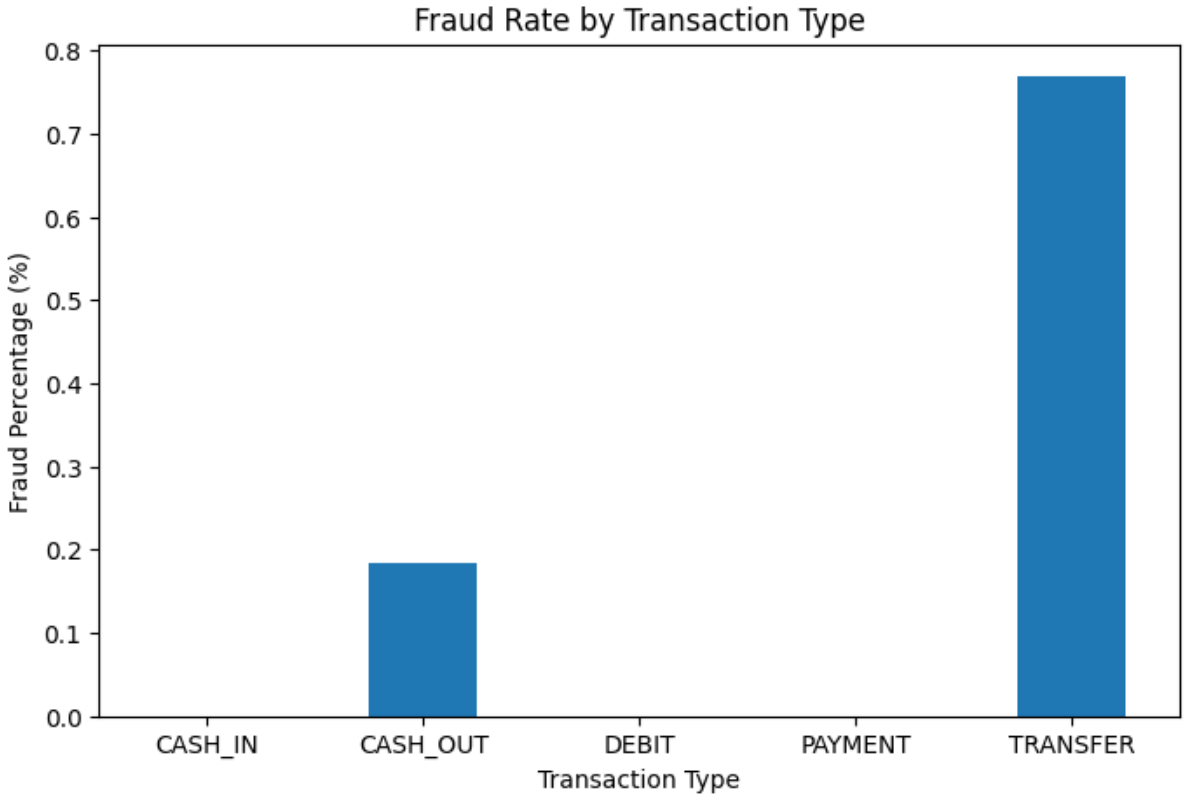
CASH_OUT 0.183955

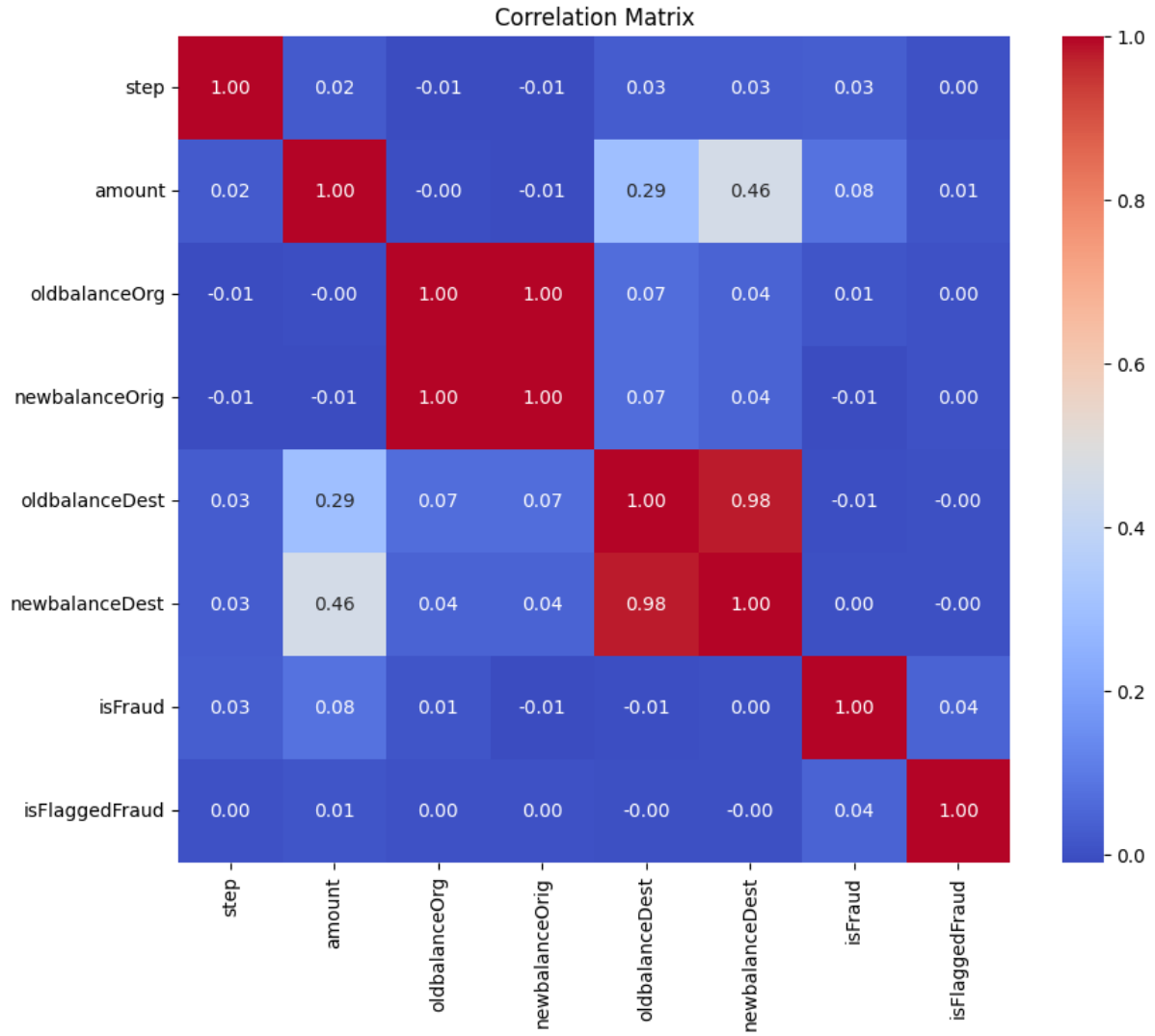
DEBIT 0.000000

PAYMENT 0.000000

TRANSFER 0.768799

Name: isFraud, dtype: float64





Fraudulent Transaction Summary:

	step	amount	oldbalanceOrg	newbalanceOrig	oldbalanceDest	newbalanceDest	isFraud	isFlaggedFraud
count	8213.000000	8.213000e+03	8.213000e+03	8.213000e+03	8.213000e+03	8.213000e+03	8213.0	8213.000000
mean	368.413856	1.467967e+06	1.649668e+06	1.923926e+05	5.442496e+05	1.279708e+06	1.0	0.001948
std	216.388690	2.404253e+06	3.547719e+06	1.965666e+06	3.336421e+06	3.908817e+06	0.0	0.044097
min	1.000000	0.000000e+00	0.000000e+00	0.000000e+00	0.000000e+00	0.000000e+00	1.0	0.000000

	step	amount	oldbalanceOrg	newbalanceOrig	oldbalanceDest	newbalanceDest	isFraud	isFlaggedFraud
25%	181.000000	1.270913e+05	1.258224e+05	0.000000e+00	0.000000e+00	0.000000e+00	1.0	0.000000
50%	367.000000	4.414234e+05	4.389835e+05	0.000000e+00	0.000000e+00	4.676420e+03	1.0	0.000000
75%	558.000000	1.517771e+06	1.517771e+06	0.000000e+00	1.478287e+05	1.058725e+06	1.0	0.000000
max	743.000000	1.000000e+07	5.958504e+07	4.958504e+07	2.362305e+08	2.367265e+08	1.0	1.000000

Normal Transaction Summary:

	step	amount	oldbalanceOrg	newbalanceOrig	oldbalanceDest	newbalanceDest	isFraud	isFlaggedFraud
count	6.354407e+06	6.354407e+06	6.354407e+06	6.354407e+06	6.354407e+06	6.354407e+06	6354407.0	6354407.0
mean	2.432357e+02	1.781970e+05	8.328287e+05	8.559702e+05	1.101421e+06	1.224926e+06	0.0	0.0
std	1.421402e+02	5.962370e+05	2.887144e+06	2.924987e+06	3.399202e+06	3.673816e+06	0.0	0.0
min	1.000000e+00	1.000000e-02	0.000000e+00	0.000000e+00	0.000000e+00	0.000000e+00	0.0	0.0
25%	1.560000e+02	1.336840e+04	0.000000e+00	0.000000e+00	0.000000e+00	0.000000e+00	0.0	0.0
50%	2.390000e+02	7.468472e+04	1.406900e+04	0.000000e+00	1.333118e+05	2.148817e+05	0.0	0.0
75%	3.340000e+02	2.083648e+05	1.069695e+05	1.447307e+05	9.441446e+05	1.111975e+06	0.0	0.0
max	7.180000e+02	9.244552e+07	4.381886e+07	4.368662e+07	3.560159e+08	3.561793e+08	0.0	0.0

==========

DATASET EXPLORATION COMPLETED

==========

Total Transactions : 6362620

Total Features : 11

Fraud Transactions : 8213

Normal Transactions: 6354407

Fraud Percentage : 0.1291 %

Algorithm Selection and Implementation

3.Feature Selection



4.PreProcessing

Features after encoding:



Shape:

(6362620, 11)

MISSING VALUES:

```
[8]
✓ 1s
X = X.replace([np.inf, -np.inf], np.nan)

X = X.fillna(0)

print("Missing values:", X.isnull().sum().sum())
```

```
Missing values: 0
```

Missing values: 0

DATA SCALING:

```
[9]
✓ 1s
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

X_scaled = scaler.fit_transform(X)

print("Scaling completed.")
print("Scaled data shape:", X_scaled.shape)
```

```
Scaling completed.
Scaled data shape: (6362620, 11)
```

Scaling completed.

Scaled data shape: (6362620, 11)

5.DBSCAN

SAMPLE FOR DBSCAN:

[10] ✓ 0s

```
# ========================================
# CREATE SAMPLE FOR DBSCAN
# ========================================

sample_size = 100000

sample_indices = df.sample(
    n=sample_size,
    random_state=42
).index

X_sample = X_scaled[sample_indices]

df_sample = df.loc[sample_indices].copy()

print("X_sample shape:", X_sample.shape)
print("df_sample shape:", df_sample.shape)
```

```
X_sample shape: (100000, 11)
df_sample shape: (100000, 11)
```

CLUSTER DISTRIBUTION:

[11] ✓ 2m

```
from sklearn.cluster import DBSCAN

dbscan = DBSCAN(
    eps=0.5,
    min_samples=10,
    n_jobs=-1
)

clusters = dbscan.fit_predict(X_sample)

df_sample['DBSCAN_Cluster'] = clusters
```

```
[12]
✓ 0s
```

```
print("Cluster distribution:")
print(
    df_sample['DBSCAN_Cluster']
    .value_counts()
    .sort_index()
)
```

```
Cluster distribution:
DBSCAN_Cluster
-1      1951
 0     21127
 1     33561
 2      7616
 3     35074
 4       557
 5        14
 6        14
 7        11
 8        16
 9         7
 10       10
 11       13
 12       10
 13       10
 14        9
Name: count, dtype: int64
```

NO OF CLUSTERS:

```
[13]
✓ 1s
```

```
n_clusters = len(set(clusters)) - (1 if -1 in clusters else 0)

n_anomalies = (clusters == -1).sum()

print("\nNumber of clusters:", n_clusters)
print("Number of anomalies:", n_anomalies)
```

```
Number of clusters: 15
Number of anomalies: 1951
```

Number of clusters: 15

Number of anomalies: 1951

EXTRACT DETECTED ANAMOLIES:

```
[14]
✓ 0s
anomalies = df_sample[
    df_sample['DBSCAN_Cluster'] == -1
]

print("Detected anomalies:", len(anomalies))

display(
    anomalies[
        [
            'step',
            'type',
            'amount',
            'oldbalanceOrg',
            'newbalanceOrig',
            'oldbalanceDest',
            'newbalanceDest',
            'isFraud'
        ]
    ].head(20)
)

Detected anomalies: 1951
```

Detected anomalies: 1951

	step	type	amount	oldbalanceOrg	newbalanceOrig	oldbalanceDest	newbalanceDest	isFraud
6066085	514	TRANSFER	1958230.60	69563.00	0.00	5345774.90	7304005.50	0
1044829	94	CASH_IN	162920.46	8643332.42	8806252.88	7520724.02	7357803.55	0
6305329	684	TRANSFER	4126119.93	344527.00	0.00	446963.89	4573083.81	0
5035616	354	TRANSFER	97301.71	31239.00	0.00	36857953.61	36955255.33	0
61015	9	CASH_IN	364810.85	2318126.43	2682937.28	9940322.17	9204546.58	0
4134957	303	CASH_OUT	43159.01	9670.00	0.00	20614518.69	20657677.70	0
6244659	596	CASH_IN	127498.89	12870344.48	12997843.38	315659.12	188160.23	0
2736446	212	TRANSFER	4953893.08	4953893.08	4953893.08	0.00	0.00	1
3751356	279	CASH_IN	165732.90	24118825.16	24284558.06	4265312.30	4099579.40	0

	step	type	amount	oldbalanceOrg	newbalanceOrig	oldbalanceDest	newbalanceDest	isFraud
6170515	563	CASH_IN	206144.99	161568.00	367712.99	14263846.91	14057701.93	0
5947508	405	CASH_IN	107475.73	1698829.23	1806304.96	51607560.24	51500084.51	0
1879235	164	CASH_IN	625810.96	812375.90	1438186.86	9534557.23	8908746.27	0
1030272	51	CASH_OUT	1170282.92	1170282.92	0.00	173264.14	1343547.06	1
3776825	280	CASH_IN	31370.08	7955056.03	7986426.11	8594084.49	8562714.41	0
5900570	403	TRANSFER	527937 2.81	2527.46	0.00	6197777.70	11477150.50	0
4598670	328	TRANSFER	500104 6.82	0.00	0.00	10039926.35	15040973.17	0
2720312	211	CASH_IN	114228.49	17101522.66	17215751.16	3452170.68	3337942.18	0
3483835	258	CASH_IN	159276.23	18393302.01	18552578.24	8427722.98	8268446.75	0
4002637	298	CASH_IN	154986.17	22069463.04	22224449.22	54785635.39	54630649.22	0
59491	9	CASH_IN	140440.07	6995392.10	7135832.18	9363168.89	9680048.95	0

DBSCAN ANAMOLY PREDICTION:

[15]
✓ 0s

```
# DBSCAN anomaly prediction
df_sample['DBSCAN_Anomaly'] = (
    df_sample['DBSCAN_Cluster'] == -1
).astype(int)

# Compare DBSCAN anomaly with actual fraud
comparison = pd.crosstab(
    df_sample['DBSCAN_Anomaly'],
    df_sample['isFraud']
)

print(comparison)
```

```
isFraud             0   1
DBSCAN_Anomaly
0               97956  93
1                1903  48
```

#precision recall and F1

[16]
✓ 0s

```
from sklearn.metrics import (
    precision_score,
    recall_score,
    f1_score
)

y_actual = df_sample['isFraud']

y_predicted = df_sample['DBSCAN_Anomaly']

precision = precision_score(
    y_actual,
    (parameter) zero_division: str
)

recall = recall_score(
    y_actual,
    y_predicted,
    zero_division=0
)

f1 = f1_score(
    y_actual,
    y_predicted,
    zero_division=0
```

```
recall = recall_score(
    y_actual,
    y_predicted,
    zero_division=0
)

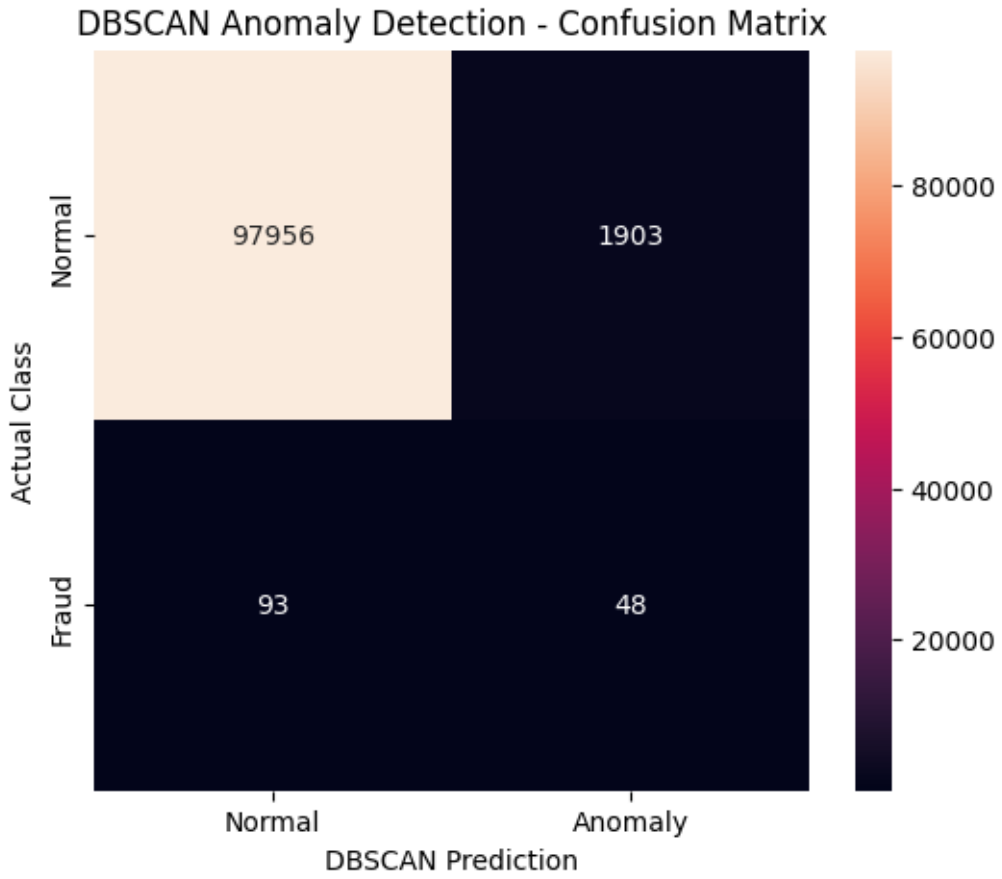
f1 = f1_score(
    y_actual,
    y_predicted,
    zero_division=0
)

print("Precision :", round(precision, 4))
print("Recall    :", round(recall, 4))
print("F1 Score  :", round(f1, 4))
```

```
Precision : 0.0246
Recall    : 0.3404
F1 Score  : 0.0459
```

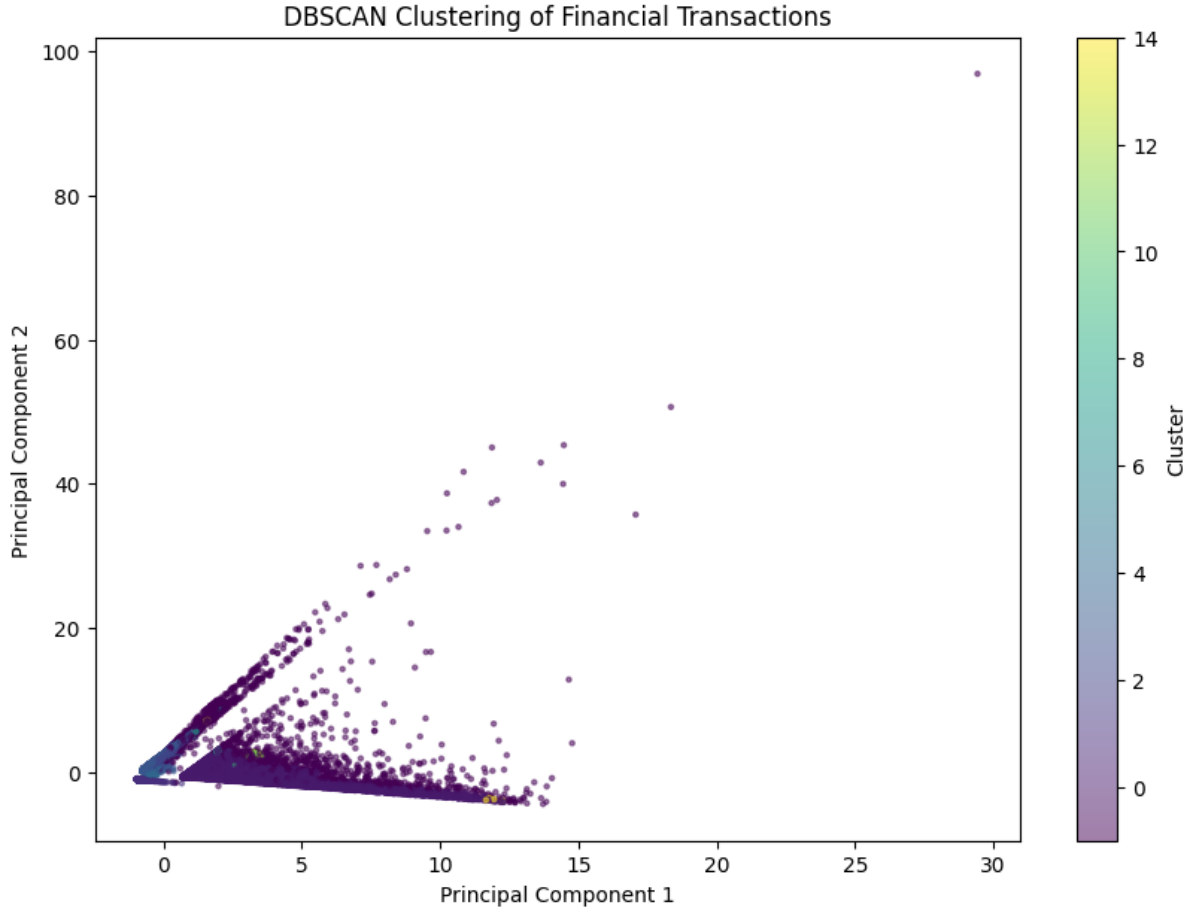
Conclusion

CONFUSION MATRIX:

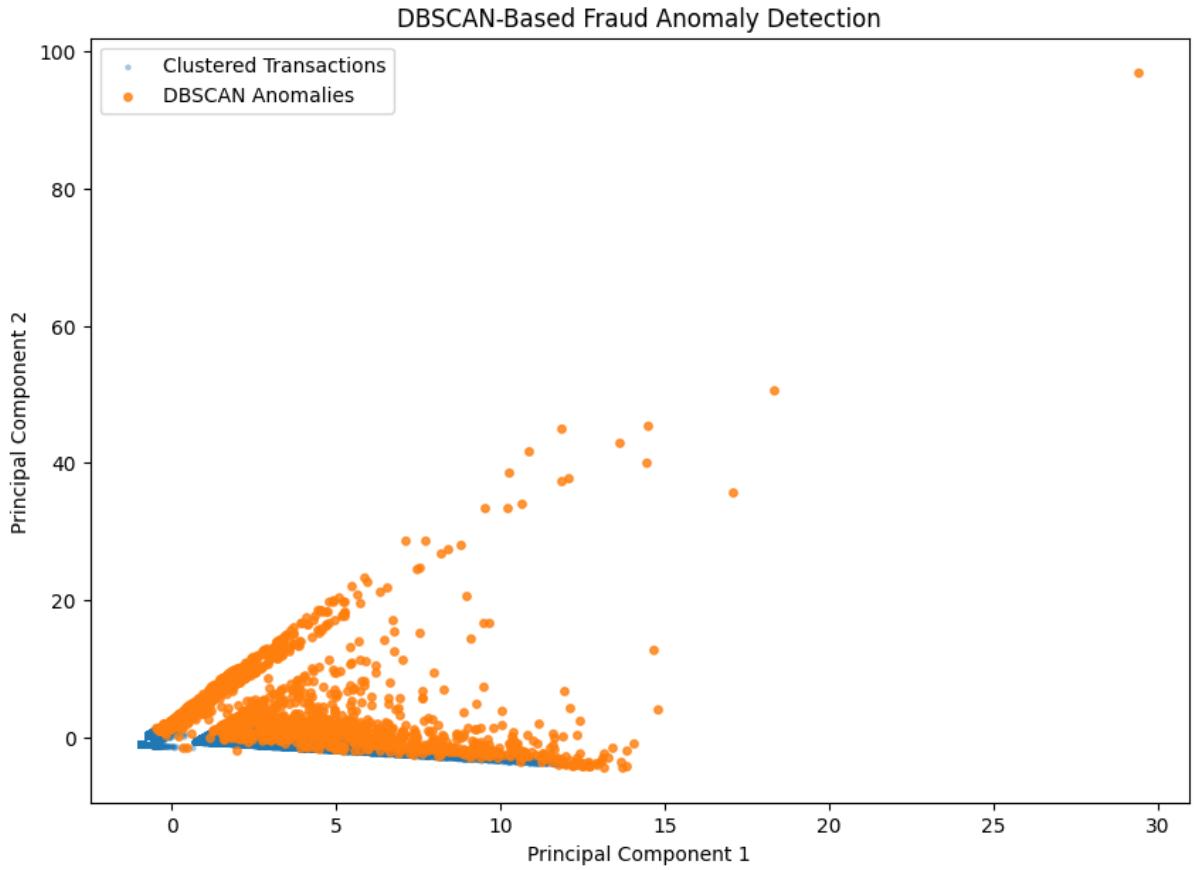


VISUALIZE DBSCAN:

PCA completed.



SHOW ANAMOLIES SEPARATELY:


DBSCAN-Based Fraud Anomaly Detection
Clustered Transactions
DBSCAN Anomalies
Principal Component 2
Principal Component 1
100
80
60
40
20
0
0
5
10
15
20
25
30